

## Sesión L06- Herencia

Para realizar esta práctica vamos a crear el directorio *L06-Herencia*, dentro de nuestro directorio de trabajo (Practicas). Por lo tanto tendremos que hacer:

```
> cd Practicas  
> mkdir L06-Herencia  
> cd L06-Herencia
```

Este será el subdirectorio en el que vamos a trabajar durante el resto de la sesión. Por lo tanto cualquier fichero o directorio que creamos en esta sesión estará dentro de *L06-Herencia*.

Copia la carpeta que os hemos proporcionado: **network-L06** en tu zona de trabajo *L06-Herencia*.

### 01

#### NewsFeed: un proveedor de noticias

Vamos a comenzar a trabajar con el proyecto **network-L06**. El proyecto está formado por tres clases. Genera la documentación javadoc usando la *Run Configuration* proporcionada. Familiarízate con el código, consultando la documentación javadoc generada y la descripción abstracta para cada una de las clases. La descripción abstracta de las clases también puedes consultarla desde vista *Outline* de Eclipse. Ya habrás podido comprobar que haciendo click en cualquier elemento de la vista *Outline* automáticamente vemos el detalle en el editor.

Crea primero la *Run Configuration* **network-L06-run**, con las siguientes goals. La usaremos para compilar y ejecutar el método *main()* de la clase *pa.Demo1*:

```
compile exec:java -Dexec.mainClass=pa.Demo1
```

Se pide:

- **Implementa** el código del método *display()* de los dos tipos de posts (*MessagePost* y *PhotoPost*, del paquete *pa.network*), para que muestre en primer lugar el nombre, mensaje y fecha del mensaje, de la siguiente forma:

```
Message author: <aquí pondremos el nombre del autor>  
Message content: <aquí pondremos el contenido del mensaje>  
Message date: <aquí mostraremos la fecha del mensaje>
```

También mostraremos:

- la fecha de creación del mensaje (invocando al método *timeString()* y pasando como parámetro el atributo *timestamp*), y
- el número de likes (si es que los tiene).
- la lista de comentarios, por orden de creación, y numerados (el primer comentario se numerará con 1). Si no hay comentarios se mostrará el mensaje "No comments."

La **Figura 1** muestra un ejemplo de salida por pantalla para ambos tipos de mensajes:

```
Message author: Mario  
Message content: Me voy a la Uni  
Message date: 0 seconds ago - 2 people like this.  
No comments.
```

Tiene 2 "likes" pero no tiene comentarios

```
Message author: Juan  
Message content: [src/main/resources/imagenes/leopardo.jpg] Leopardo enfadado  
Message date: 0 seconds ago  
2 comment(s):  
1. Qué foto más chula!  
2. Pues a mí no me gusta mucho
```

El mensaje de Juan es de tipo PhotoPost

En este caso el mensaje no tiene "likes" pero sí tiene comentarios

**Figura 1.** Salida por pantalla de la ejecución del método *display()* para los dos tipos de mensajes

- **Implementa** el código del método `pa.network.Newsfeed.show()` para que muestre por pantalla los mensajes de ambos tipos (de texto y de imagen). Deberás usar los métodos `display()` que has implementado en el apartado anterior.
- Crea una nueva clase **`pa.Demo1`** con un método **`main()`**. En dicho método debes crear una instancia de **`NewsFeed`** que contenga dos objetos, uno de cada tipo de post.
- Para la instancia de tipo `MessagePost` formato texto añade 3 *likes* y un *unlike* (en realidad no contabilizamos los "unlikes", el método `unlike()` lo que hace es retirar un *like*). Para el objeto de tipo `PhotoPost` puedes usar alguna de las imágenes que encontrarás en `src/main/resources/imagenes`. Para añadir una imagen al post debes usar su ruta completa, por ejemplo la imagen `imagen.jpg` debes añadirla como `"src/main/resources/imagenes/imagen.jpg"`. Para el objeto de tipo `PhotoPost` añade 2 comentarios. En la **Figura 2** mostramos un ejemplo de ejecución.
- Ahora muestra los dos mensajes (invocando al método `Newsfeed.show()`).
- A continuación añade al primer mensaje (de tipo texto) 7 comentarios diferentes, 18 *likes*, y 25 *unlikes*. Para el segundo (mensaje de tipo foto), añade 10 comentarios diferentes, 35 *likes*, y 4 *unlikes*. DEBES utilizar obligatoriamente sentencias de bucle. Finalmente visualiza de nuevo el contenido del objeto de tipo `NewFeeds` invocando su método `show()`. En la **Figura 2** mostramos un ejemplo de ejecución.

```

Message author: Mario
Message content: Me voy a la Uni
Message date: 0 seconds ago - 2 people like this.
No comments.

Message author: Juan
Message content: [src/main/resources/imagenes/leopardo.jpg] Leopardo enfadado
Message date: 0 seconds ago
2 comment(s).
1. Qué foto más chula!
2. Pues a mí no me gusta mucho

Message author: Mario
Message content: Me voy a la Uni
Message date: 0 seconds ago
- 7 comment(s):
1. comentario1
2. comentario2
3. comentario3
4. comentario4
5. comentario5
6. comentario6
7. comentario7

Message author: Juan
Message content: [src/main/resources/imagenes/leopardo.jpg] Leopardo enfadado
Message date: 0 seconds ago - 31 people like this.
12 comment(s).
1. Qué foto más chula!
2. Pues a mí no me gusta mucho
3. comentarioPhoto1
4. comentarioPhoto2
5. comentarioPhoto3
6. comentarioPhoto4
7. comentarioPhoto5
8. comentarioPhoto6
9. comentarioPhoto7
10. comentarioPhoto8
11. comentarioPhoto9
12. comentarioPhoto10

```

Mensaje de texto con dos likes (inicialmente tenía 3 likes, pero hemos retirado uno)

Mensaje de tipo foto sin likes y con 2 comentarios

Añadimos 7 comentarios diferentes al primer mensaje

Añadimos 10 comentarios diferentes, 18 likes y 25 unlikes al segundo mensaje

**Figura 2.** Salida por pantalla de la ejecución del método `pa.Demo1.main()`. Todo lo que se muestra en la pantalla está generado por la invocación a los métodos que corresponda

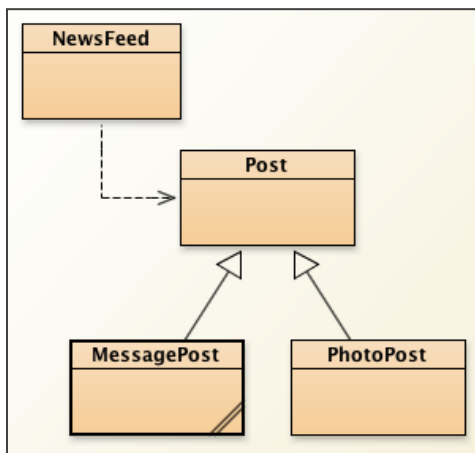
Sube GitHub el trabajo realizado hasta el momento:

```
> git add . //desde el directorio que contiene el repositorio git
> git commit -m"Terminado el ejercicio 1 de L06: network"
> git push
```

## 02

### Mejoramos el código introduciendo herencia

Vamos a crear una nueva versión del proyecto anterior. En concreto vamos a realizar modificaciones en el código para que su diagrama de clases sea como el que se muestra en la **Figura 3**.



La clase **NewsFeed** utiliza objetos de tipo *Post* (flecha con línea discontinua). Por lo tanto la clase *NewsFeed* "depende" de la clase *Post*.

La clase **Post** es superclase de **MessagePost** y **PhotoPost**. Recuerda que la superclase define los comportamientos comunes a todas sus subclases.

Cada subclase "hereda" los comportamientos de su superclase (también denominada "clase padre"), de forma que puede utilizar el comportamiento por defecto (el que ha heredado), o bien sobrescribir dicho comportamiento y "particularizarlo" para dicha subclase.

Recuerda que para sobrescribir un método usaremos la anotación `@Override`.

**Figura 3.** Diagrama de clases final

Vamos a crear un nuevo proyecto maven al que llamaremos **herencia-L06**. Lo haremos desde el terminal o desde el administrador de archivos de linux:

- Crea una copia de la carpeta *network-L06* (del ejercicio anterior) en tu directorio de trabajo y cámbiale el nombre a **herencia-L06**. Si lo haces desde el terminal puedes usar el comando `cp -r network-L06 herencia-L06`
- Edita el fichero **pom.xml** (de *herencia-L06*) y cambia el valor de la etiqueta `<artifactId>` por *herencia-L06*. Debe quedar como `<artifactId>herencia-L06</artifactId>`
- Borra los ficheros y directorios siguientes. (OJO: desde *herencia-L06*!!). Puedes hacerlo de forma manual (uno a uno), o puedes desde el terminal simplemente ejecutar el script que os hemos proporcionado.
  - Si usas el script, simplemente ejecuta desde el terminal: `./borrarFicheros.sh`
  - Si prefieres el proceso manual, debes borrar todos los ficheros ocultos que ha creado eclipse (ficheros `.classpath`, `.project` y el directorio `.setings`). Borra, además, todos los ficheros con extensión `.launch` y el directorio `target`.
- finalmente importa el nuevo proyecto (*herencia-L06*) desde eclipse

Crea la *Configuration* **herencia-L06-clean** (en la subcarpeta "**eclipse-launch-files**", igual que hemos hecho en prácticas anteriores), con la goal: `clean`

Crea la *Run Configuration* **herencia-L06-run** (en la subcarpeta "**eclipse-launch-files**", igual que hemos hecho en prácticas anteriores), con los comandos:

```
compile exec:java -Dexec.mainClass=pa.Demo2
```

Una vez que tenemos el nuevo proyecto **herencia-L06**, se pide:

- Crea una nueva clase **Post**, que sea "padre" de los dos tipos de mensajes, tal y como se muestra en la **Figura 3**. Para implementar la clase *Post* debes tener en cuenta la representación abstracta mostrada en la **Figura 4**.

En la **Figura 4** se muestran en rojo los constructores de las clases. Fíjate que los campos de *Post* son **protected** (indicado con el símbolo "#", lo cual significa que no serán accesibles desde fuera de la clase *Post*, pero sí lo serán para cualquiera de sus subclases, en este caso *MessagePost* y *PhotoPost*. La clase *Post* tiene también un método **protected**.

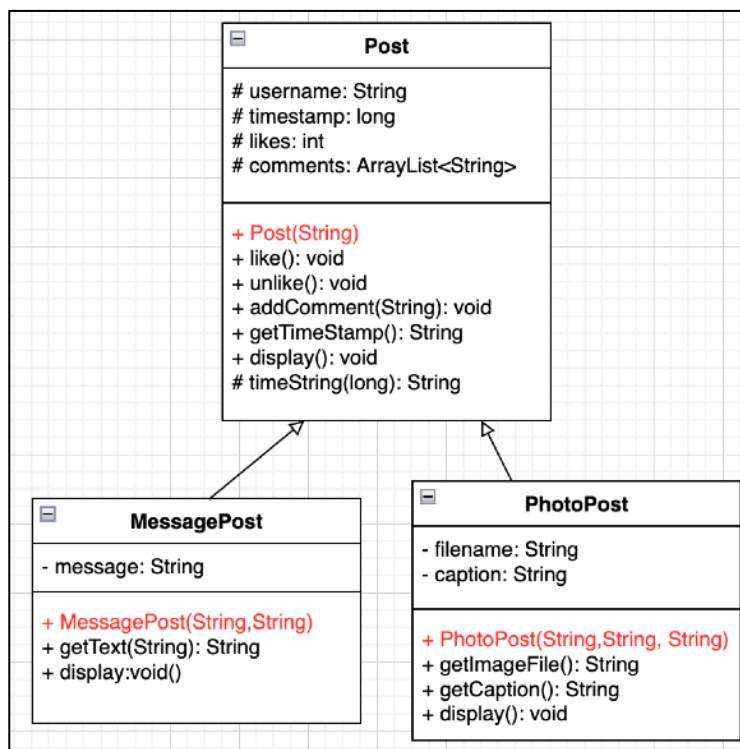
**Figura 4.** Jerarquía de clases *Post*

Observa también que la clase *MessagePost* tiene el método *getText()*, que es particular de dicha clase.

De igual forma, los métodos *getImageFile()* y *getCaption()* son exclusivos de los objetos de tipo *PhotoPost*.

Las subclases *MessagePost* y *PhotoPost* sobrescriben el método *display()* de la clase padre (*Post*). Recuerda que usar la anotación **@Override** sobre el método que vas a sobrescribir.

**Nota:** Fíjate que si las clases hijas no sobrescriben el método *display()*, éste no aparecerá en la descripción abstracta de dichas clases hijas.



- Modifica el código de las clases **MessagePost** y **PhotoPost** convenientemente para que se correspondan con la representación abstracta mostrada en la **Figura 4**. Las subclases de *Post* sobrescriben el método *display()*, de forma que cada una lo implementará de forma diferente, dependiendo del tipo de mensaje.
- Finalmente modifica el código de la clase **NewsFeed** para que dependa únicamente de la clase *Post*, tal y como se muestra en la **Figura 3**. Esto significa que en el código de *NewsFeed* NO puedes utilizar ningún objeto de tipo *MessagePost* y/o *PhotoPost*. Por lo tanto deberás modificar tanto los **campos** de la clase, como el código del método **show()**, y también sustituir los dos métodos *addPhotoPost()* y *addMessagePost()* por un único método con nombre **addPost()**.
- Añade una nueva clase denominada **Demo2** con un método *main()* con el mismo código que la clase del ejercicio anterior (*Demo1*), pero tendrás que realizar las modificaciones oportunas para que funcione exactamente igual que antes. Borra el fichero *Demo1.java*.
- Finalmente observa que podemos crear también objetos de tipo *Post* y añadirlos a nuestra instancia de *NewsFeed*, pero en ese caso NO podemos mostrar por pantalla el mensaje asociado, ya que los objetos de tipo *Post* no disponen de ningún campo para almacenar esta información, únicamente podremos mostrar comentarios, y los "likes" de los objetos creados a partir de *new Post()*. Compruébalo creando una instancia de tipo *Post*, añade 5 comentarios, 17 likes y 13 unlike y finalmente vuelve a invocar al método *NewsFeed.show()*.

Sube GitHub el trabajo realizado hasta el momento:

```
>git add . //desde el directorio que contiene el repositorio git
>git commit -m"Terminado el ejercicio 2 de L06: herencia"
>git push
```

## 03

### Trabajamos con clases abstractas

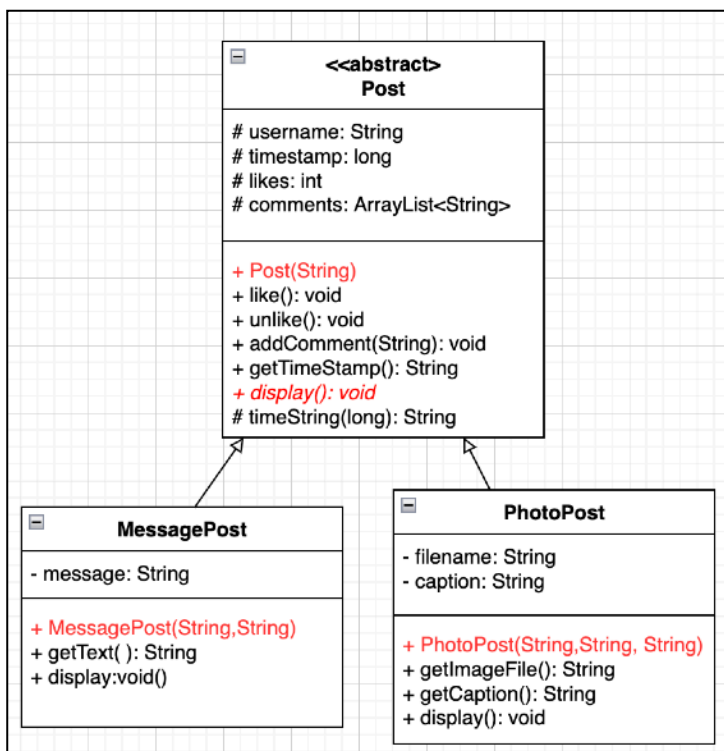
Ahora vamos a seguir mejorando nuestro código, y vamos a introducir una clase abstracta. Vuelve a **duplicar** el proyecto *herencia-L06* (igual que hemos hecho en el ejercicio anterior), y esta vez cámbiale el nombre a la copia a **abstract-L06**. Tendrás que volver a cambiar el nombre del `<artifactId>` en el `pom.xml`. También que ejecutar el script *borrarFicheros.sh*, o bien borrar todos los ficheros innecesarios manualmente.

Sobre el nuevo proyecto *abstract-L06*, se pide:

- Crea las siguientes **Run Configuration** en la subcarpeta "*eclipse-launch-files*":
  - **abstract-L06-clean**: comando: `clean`
  - **abstract-L06-run**: comandos: `compile exec:java -Dexec.mainClass=pa.Demo3`
- Modifica el código de la clase *Post* de forma que sea una clase abstracta, y convierte el método *display()* en abstracto. La **Figura 5** muestra la jerarquía de clases para el proyecto **abstract-L06**.

Al final del ejercicio anterior hemos indicado que no es práctico crear instancias de tipo *Post*, puesto que al carecer de un campo que represente el mensaje solamente podemos añadir comentarios, sin hacer referencia a ningún mensaje. Esto no tiene mucho sentido.

Si convertimos la clase *Post* en abstracta, precisamente evitaremos esta situación.



Recuerda que NO podemos crear instancias a partir de una clase abstracta (parece lógico, ya que alguno de sus métodos NO está implementado, en este caso el método *display()*).

Para expresar que un método es "abstracto" lo indicaremos poniendo el nombre el **cursiva** en el diagrama de clases.

**Figura 5.** Jerarquía de clases Post



- Vamos a realizar algunas modificaciones en la implementación del método **display()** para cada subclase de *Post*. Comenzamos con la implementación de **MessagePost.display()**. Realiza las modificaciones oportunas para que muestre por pantalla la siguiente información de un post de tipo *MessagePost* (ver **Figura 6**).

```
Gloria
Text Message: Me gustó la peli de anoche
0 seconds ago
2 comment(s).
Anónimo dice :A mi también
Anónimo dice :Pues a mí no
```

Esta salida muestra el mensaje de Gloria "Me gustó la peli de anoche".

Cada comentario tiene que ir precedido de " Anónimo dice :"

**Figura 6.** Salida del método *MessagePost.display()*

- Modifica el código de *PhotoPost.display()*. La idea es mostrar por pantalla, además de los comentarios, la imagen enviada. Tendremos que mostrarla en una nueva ventana. Para ello añade el siguiente método a la clase *PhotoPost* indicado en la **Figura 7**. Tendrás que invocar a dicho método desde *display()*

```
import javax.swing.JFrame;
import javax.swing.ImageIcon;
import javax.swing.JLabel;
...

public void showImage(String filename, String caption, String
username) {
    JFrame frame = new JFrame(caption+ " (de "+username+"");
    ImageIcon icon = new ImageIcon(filename);
    JLabel label = new JLabel(icon);
    frame.add(label);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.pack();
    frame.setVisible(true);
}
```

**Figura 7.** Código para mostrar la imagen en una nueva ventana

El nombre de los ficheros (parámetro *filename*) deberá incluir la ruta para llegar a ellos, por ejemplo "src/main/resources/imagenes/nombre\_fichero.jpeg".

En este caso deberías obtener por pantalla el resultado que mostramos en la **Figuras 8 y 9**.

```
Roberto
This is a Photo Message. See the opened window. Remember close that window
2 seconds ago
No comments.
```

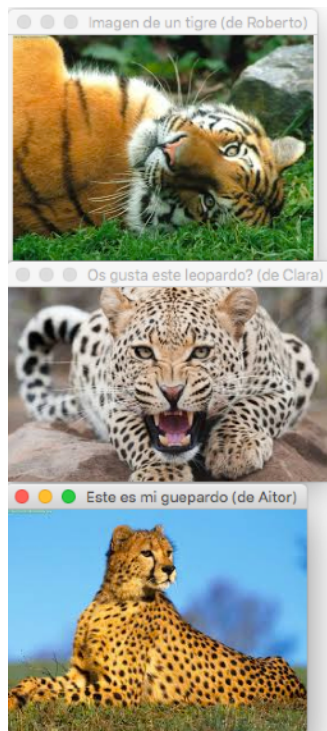
**Figura 8.** Salida por pantalla de un post de tipo *PhotoPost* de Roberto. Mostraremos el mensaje que ves en la segunda línea. En este ejemplo, el post no tiene comentarios



**Figura 9.** Imagen de Roberto, con el *caption* "Imagen de un tigre"

- Implementa una clase **pa.Demo3** para probar todos los cambios de los apartados anteriores. Esta clase implementará un método *main()* en el que crearemos cuatro posts:
  - ❖ El primero de tipo *PhotoPost*, de Roberto, con la imagen del tigre y el *caption* "Imagen de un tigre", y sin comentarios.
  - ❖ El segundo también de tipo *PhotoPost*, de Clara, con el *caption* "Os gusta este leopardo?", y con un comentario "Qué chulo!!".
  - ❖ El tercero de tipo *PhotoPost*, de Aitor, con la imagen del guepardo y el *caption* "Este es mi guepardo", sin comentarios y con 20 likes.
  - ❖ El cuarto de tipo *MessagePost*, de Gloria, con el texto "Me gustó la peli de anoche", y con dos comentarios ("A mi también", y "Pues a mí no").

En la **Figura 10** mostramos el resultado de ejecución de *pa.Demo3.main()*:



```
Roberto
This is a Photo Message. See the opened window.
Remember close that window
2 seconds ago
No comments.

Clara
This is a Photo Message. See the opened window.
Remember close that window
6 seconds ago
1 comment(s).
Anónimo dice: Qué chulo!!

Aitor
This is a Photo Message. See the opened window.
Remember close that window
0 seconds ago - 20 people like this.
No comments.

Gloria
Text Message: Me gustó la peli de anoche
0 seconds ago
2 comment(s).
Anónimo dice :A mi también
Anónimo dice :Pues a mí no
```

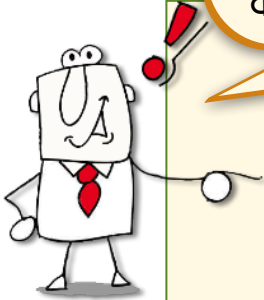
**Figura 10.** Resultado final con todas las modificaciones del ejercicio

- Borra el fichero *Demo2.java*

Sube GitHub el trabajo realizado hasta el momento:

```
>git add . //desde el directorio que contiene el repositorio git
>git commit -m"Terminado el ejercicio 3 de L06: abstract"
>git push
```

¿Qué **conceptos** y **cuestiones** me deben quedar CLAROS después de hacer la práctica?



- La herencia y las clases abstractas nos permiten estructurar mejor nuestro código EVITANDO DUPLICACIONES innecesarias
- La herencia y las clases abstractas nos permiten REUTILIZAR código y facilitar los CAMBIOS (reduciendo el tiempo necesario para realizarlos)
- La herencia determina una relación "es-un" y se diferencia claramente de la relación de dependencia entre clases "usa"
- La relación de herencia permite **compartir atributos y comportamientos**. Si dos clases no tienen atributos en común, entonces no podemos usar una relación de herencia entre ellas.
- La clase "padre" no tiene por qué compartir todos sus atributos y comportamientos con sus hijos. Puede tener atributos y/o métodos privados.
- Cualquier clase en Java tiene como mucho una única clase "padre"
- Dado un problema, tenemos que saber decidir qué relación entre clases es mejor: el uso de dependencia o de herencia
- Tenemos que tener claro cuándo conviene usar una relación de herencia o el uso de clases abstractas
- Entender y saber usar el polimorfismo (de variables y métodos) a nivel de código